Nombre: Jose Gerardo Ortiz Leñero	**Matrícula**: AL07182099
Nombre del curso: Gestión de proyectos de inteligencia artificial **Grupo:** CC.MSCA2102FLE.102.202630	**Nombre del profesor**: Ing. Tania Judith Mendoza Torres
Módulos: 15. Despliegue local. 16. Despliegue en la nube.	**Actividad**: Fase II. Manual de despliegue proyecto DEMANDA-IA.
Fecha: 01/Nov/2026	
MASTER INTELIGENCIA ARTIFICIAL	

Universidad Tecmilenio

Tabla de Contenido

1.- Propósito y alcance 3
2.- Arquitectura de la solución 3
3.- Requerimientos técnicos 5
4.- Estructura del repositorio 6
5.- Despliegue local paso a paso 6
6.- Estrategia de despliegue en la nube 8
7.- Despliegue en Google Cloud Run paso a paso 9
8.- Operación y mantenimiento 12
9.- Alineación con ISO/IEC 23053 14
10.- Uso de herramientas de IA generativa en la documentación 14
Bibliografía 15

1.- Propósito y alcance

Este documento explica cómo construir, ejecutar y publicar **DEMANDA-IA**, el servicio que pronostica la demanda diaria por SKU y sucursal para Farmacias Occidente Salud. En la Fase I definí el caso de negocio, una red de 140 sucursales en el occidente de México que hoy reabastece con reglas fijas y que quiere pasar a un pedido calculado cada noche. En esta Fase II el modelo ya existe como servicio y este documento es la guía paso a paso para levantar el servicio.

Cubre la arquitectura y sus componentes, los requerimientos técnicos, el procedimiento paso a paso (primero en local con Docker y después en la nube) y la operación diaria una vez publicado. Las pruebas y sus resultados están en un documento aparte, el Documento de validación y pruebas.

Nota: El repositorio usa datos sintéticos generados con un simulador que reproduce los patrones descritos en la Fase I. El procedimiento de despliegue es el mismo con datos reales. Lo único que cambia es el origen del historial de ventas, que en producción debería venir del ERP.

2.- Arquitectura de la solución

La arquitectura seleccionada fue de un solo contenedor con dos caras. Una API REST para sistemas (el ERP y la corrida nocturna) y una interfaz web para personas (gerentes de sucursal o el área de compras). Las dos viven en la misma imagen porque comparten el modelo y así se evita tener que configurar CORS y dos despliegues para un proyecto de este tamaño. Si en un futuro el frontend crece, se puede separar sin tocar el backend. El archivo `app.js` ya acepta una URL de API distinta.

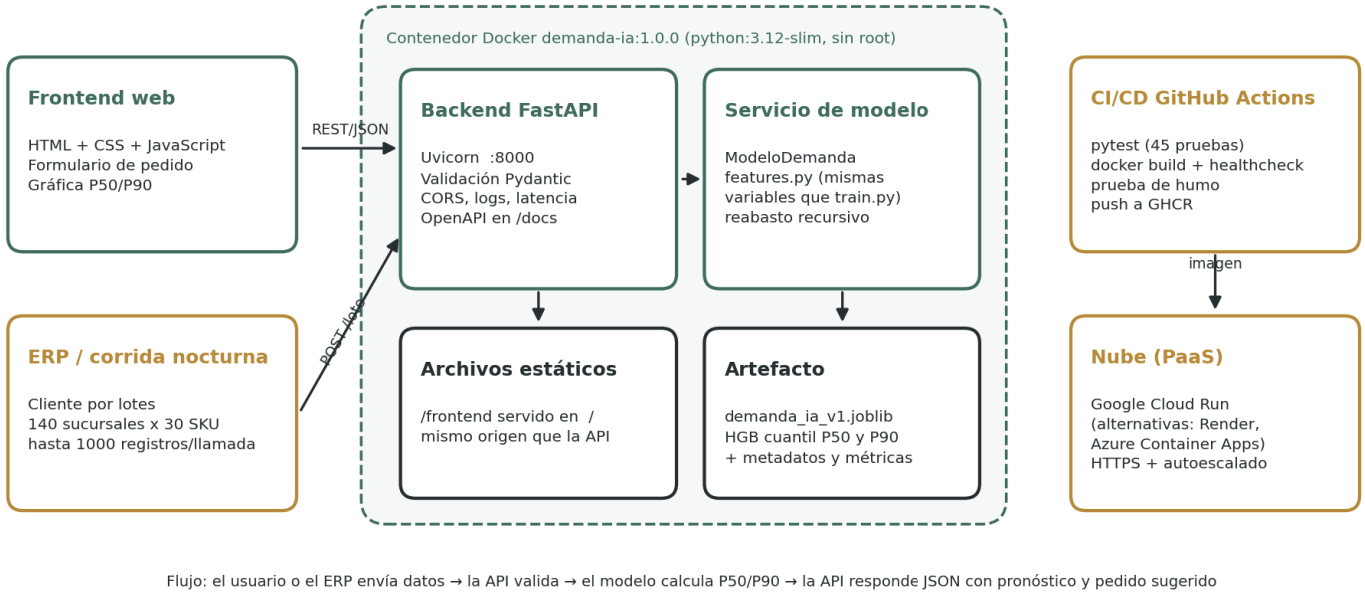


Figura 1. Arquitectura de DEMANDA-IA: clientes, contenedor, CI/CD y nube.

Componente	Ubicación en el repo	Responsabilidad
Simulador de datos	src/demanda_ia/data.py	Genera ventas diarias con estacionalidad, quincenas, festivos, promociones y quiebres de stock
Ingeniería de variables	src/demanda_ia/features.py	Arma las 15 variables de entrada. La usan el entrenamiento y la API, así no hay diferencias entre ambos
Entrenamiento offline	src/demanda_ia/train.py	Entrena los modelos P50 y P90, evalúa y guarda el artefacto .joblib con sus métricas

Servicio de modelo	src/demanda_ia/model.py	Carga el artefacto, revisa entradas atípicas y calcula pronósticos y pedido sugerido
Contratos de la API	src/demanda_ia/schemas.py	Valida tipos, rangos y catálogos con Pydantic; responde 422 si algo no cuadra
Backend REST	src/demanda_ia/api.py	FastAPI con 7 endpoints, log de latencia, CORS configurable y documentación OpenAPI
Frontend	frontend/	Página HTML/CSS/JS sin librerías externas que consume la API
Contenedor	Dockerfile, docker-compose.yml	Imagen multi-etapa, usuario sin root, healthcheck
CI/CD	.github/workflows/ci.yml	Pruebas, build, healthcheck, prueba de humo y publicación en GHCR

Tabla 1. Componentes del sistema.

Modelo

El modelo son dos regresores HistGradientBoostingRegressor de scikit-learn entrenados con pérdida de cuantiles. El primero estima el cuantil 50 (la venta típica del día) y el segundo el cuantil 90 (un día alto). Con los dos se calcula el pedido sugerido: la demanda esperada del periodo (suma de los P50 diarios) más un stock de seguridad, menos la existencia en anaquel, redondeado hacia arriba y con mínimo cero. El stock de seguridad es la raíz cuadrada de la suma de los cuadrados de (P90 − P50) de cada día del horizonte, suponiendo que los errores diarios son independientes.

Las variables de entrada son de tres tipos. Atributos fijos (categoría del producto, tipo de sucursal, estado, tamaño de la tienda, demanda base del SKU), calendario y precio (día de la semana, mes, día del mes, quincena, festivo, precio relativo a lista, promoción) e historial reciente (venta de hace 7 días, promedio de 7 días y promedio de 28 días). Como el modelo usa atributos de la sucursal y no su identificador, funciona también en sucursales que no estaban en el entrenamiento. Esto se comprobó con 10 sucursales apartadas.

Contrato de la API

Método	Entrada	Salida
GET /salud	Ninguna	Estado, modelo cargado, versión, segundos activo.
GET /modelo	Ninguna	Algoritmo, variables, periodo de entrenamiento y métricas.
GET /catalogo	Ninguna	140 sucursales, 30 SKUs y límites de uso.
GET /historial	sucursal_id, sku, hasta, dias	Ventas diarias simuladas (solo para la demo).
POST /pronostico	sucursal_id, sku, fecha, lag_7, media_7, media_28, precio y promoción opcionales	P50, P90, unidades sugeridas, advertencias.
POST /pronostico/lote	Lista de 1 a 1000 solicitudes como la anterior	Lista de resultados y tiempo de cómputo.
POST /reabasto	sucursal_id, sku, fecha_inicio, horizonte_dias, existencia_actual y historial_ventas y también acepta precio y dias_promocion opcionales.	Pronóstico por día, demanda esperada, stock de seguridad y pedido sugerido.

Tabla 2. Endpoints expuestos (prefijo /api/v1).

Además, `/docs` muestra la documentación interactiva (Swagger) generada por FastAPI, y / sirve la interfaz web. Toda respuesta incluye el encabezado `X-Tiempo-Respuesta-ms`, útil para monitorear desde fuera.

3.- Requerimientos técnicos

Ejecución en entorno local

Herramienta	Versión probada o mínima	Objetivo
Docker Desktop (Windows/macOS) o Docker Engine (Linux)	Docker 24 o superior, con BuildKit	Construir y correr la imagen
Git	2.40 o superior	Clonar el repositorio
Navegador web	Cualquiera reciente	Interfaz y Swagger
curl o PowerShell 5+	El que traiga el sistema	Prueba de humo
Python (opcional)	3.12	Correr pruebas o reentrenar fuera de Docker
Visual Studio Code (opcional)	Cualquiera reciente	Editar y usar la terminal integrada, como en el tutorial del Tema 15

Tabla 3. Software necesario para ejecutar desde una máquina de trabajo local.

En Windows, Docker Desktop necesita WSL 2 activado. La máquina debería tener al menos 8 GB de RAM y unos 2 GB libres en disco para la imagen base y las capas

Dependencias de la imagen

Las versiones están fijas en `requirements.txt`. Esto es importante porque un archivo `.joblib` de scikit-learn solo se garantiza compatible con la misma versión con la que se guardó. Si se cambia la versión de scikit-learn en requirements.txt sin reentrenar, el contenedor puede arrancar con advertencias o fallar al cargar el modelo.

Paquete	Versión	Uso
Python (imagen python:3.12-slim)	3.12	Intérprete
fastapi / starlette	0.162.0 / 1.10.0	Framework web
pydantic	2.14.1	Validación de entradas y salidas
uvicorn	0.54.0	Servidor ASGI
scikit-learn	1.8.0	Modelos de cuantiles
numpy / scipy / joblib	2.4.4 / 1.17.1 / 1.5.3	Cómputo y carga del artefacto

Tabla 4. Dependencias de ejecución.

pandas, pytest, httpx y matplotlib están en requirements-dev.txt. Solo se usan para entrenar, probar y generar reportes, por eso no se instalan en la imagen final.

Recursos medidos

Con el servicio corriendo con dos workers (WEB_CONCURRENCY=2. La imagen usa 1 por defecto) se midió unos 97 MB de memoria por worker y 230 MB en total. El artefacto del modelo pesa 1.8 MB. Un pronóstico individual tarda alrededor de 10 ms y un lote de 1000 registros unos 50 ms. La corrida nocturna completa (140 sucursales por 30 SKU, 4 200 pronósticos) se resolvió en 0.22 segundos. Con esos números, una instancia de 1 vCPU y 512 MB a 1 GB de memoria alcanza de sobra.

Ejecución en entorno en la nube

- Cuenta de Google Cloud con facturación activa y la CLI gcloud instalada (opción principal).
- Como alternativa, una cuenta de Render o de Azure. Los planes gratuitos y sus límites cambian seguido, así que conviene revisarlos el día que se despliegue.

Cuenta de GitHub con Actions habilitado. El flujo publica la imagen en GitHub Container Registry (GHCR) con el token que GitHub da a cada ejecución, sin configurar secretos adicionales.

4.- Estructura del repositorio

```
demanda-ia/
├── .github/workflows/ci.yml   # pruebas, build, healthcheck, humo y publicación en GHCR
├── data/                      # vacío (.gitkeep); datos generados en memoria
├── docs/                      # manual de despliegue y documento de validación
├── frontend/                  # index.html, styles.css, app.js
├── models/                    # demanda_ia_v1.joblib y demanda_ia_v1.json (metadatos)
├── reports/                   # métricas, sensibilidad, resultados de pruebas y figuras
├── scripts/                   # smoke_test.sh/.ps1, benchmark.py, generar_reportes.py,
diagrama_arquitectura.py
├── src/demanda_ia/            # config, data, features, train, model, schemas, api
├── tests/                     # test_api_funcional, test_casos_extremos, test_modelo
├── Dockerfile                 # imagen multi-etapa
├── docker-compose.yml
├── .dockerignore, .gitignore, .gitattributes, .env.example
├── requirements.txt           # dependencias de ejecución (versiones exactas)
├── requirements-dev.txt       # entrenamiento, pruebas y reportes
└── README.md
```

El repositorio está publicado en https://github.com/gerardortizle/demanda-ia. El README tiene la versión corta de este manual.

5.- Despliegue local paso a paso

Los comandos a continuación deben escribirse en la terminal de Visual Studio Code

a) Obtener el código

Clonar el repositorio de GitHub del proyecto DEMANDA-IA :

```
git clone https://github.com/gerardortizle/demanda-ia.git
cd demanda-ia
```

b) Revisar el Dockerfile

El Dockerfile sigue los pasos 3 a 8 del tutorial del Tema 15 del curso (imagen base, directorio de trabajo, copiar archivos, instalar dependencias, exponer puerto y comando de ejecución) y agrega cuatro mejoras que se explican a continuación:

- **Tres etapas.** `dependencias` instala los paquetes en un entorno virtual. `pruebas` es opcional: corre pytest solo cuando se construye con `docker build --target pruebas -t demanda-ia:test .`; el build normal la omite porque la etapa final no depende de ella, y en el CI las pruebas corren en un job previo. *final* copia solo el entorno virtual y el código. La imagen final no lleva compiladores, pruebas ni pandas.

- **Usuario sin privilegios**. El proceso corre como `appuser` (uid 10001).
- **HEALTHCHECK.** Cada 30 segundos Docker llama a `/api/v1/salud`. Si falla tres veces seguidas, el contenedor queda marcado como unhealthy.
- Puerto configurable. El comando usa la variable PORT (8000 por defecto). Plataformas como Render o Cloud Run pueden asignar otro puerto y la imagen se adapta sin cambios.

c) Construir la imagen

```
docker build -t demanda-ia:1.0.0 .
```

La primera vez tarda unos minutos porque descarga la imagen base e instala scikit-learn. Al final debe aparecer `naming to docker.io/library/demanda-ia:1.0.0`. Para confirmar que la imagen existe, correr este comando:

```
docker images demanda-ia
```

d) Ejecutar el contenedor

```
docker run -d --name demanda -p 8000:8000 demanda-ia:1.0.0
docker ps
```

El parámetro `-d` deja el contenedor en segundo plano y `-p 8000:8000` conecta el puerto 8000 de la máquina con el del contenedor. Entre 30 y 60 segundos después, docker ps debe mostrar (healthy) en la columna STATUS.

e) Verificar que funciona

Primero la prueba de humo, que verifica seis endpoints de la API (el séptimo, /historial, se consulta para armar la solicitud de reabasto), la interfaz web, Swagger y un caso inválido que debe responder 422. Termina con error si algo no responde como debe:

```
# Linux, macOS o Git Bash
./scripts/smoke_test.sh http://localhost:8000

# Windows (PowerShell o terminal de VSCode)
powershell -ExecutionPolicy Bypass -File .\scripts\smoke_test.ps1
```

La salida esperada termina en `Prueba de humo superada`.

Posteriormente, en el navegador se puede observar el front end ejecutándose:

URL	Qué debe verse
http://localhost:8000/	Interfaz de DEMANDA-IA con la etiqueta verde «API en línea · modelo v1.0.0»
http://localhost:8000/docs	Swagger con los endpoints agrupados en operación, catálogo e inferencia
http://localhost:8000/api/v1/salud	JSON con "estado": "ok" y "modelo_cargado": true

Tabla 5. Direcciones de verificación.

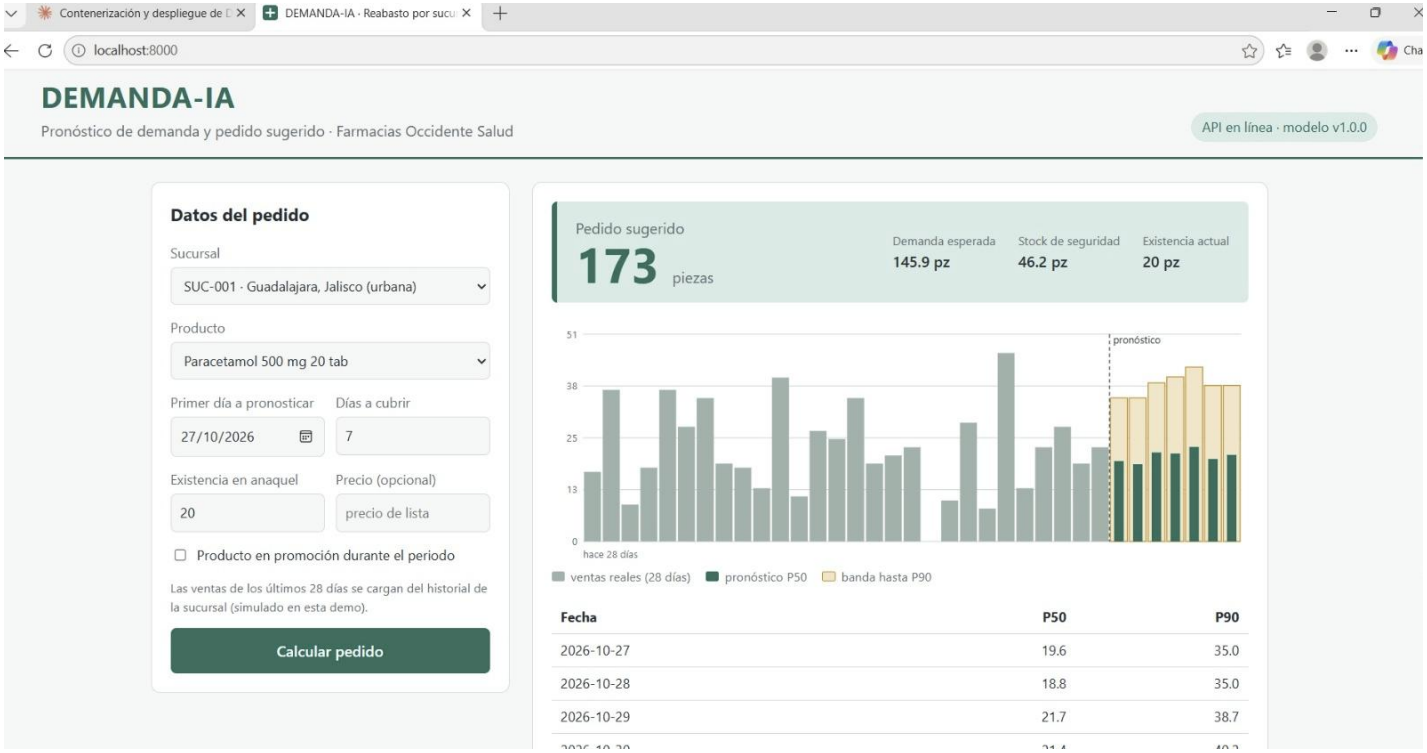


Figura 2. Interfaz web consumiendo /api/v1/historial y /api/v1/reabasto: Paracetamol en SUC-001, 7 días, pedido sugerido de 173 pz.

f) Detener y limpiar

```
docker rm -f demanda
```

6.- Estrategia de despliegue en la nube

Modelo de servicio

Comparé las cuatro opciones que fueron estudiadas en el Tema 16 (IaaS, PaaS, SaaS y FaaS) pensando en lo que DEMANDA-IA requiere. Se necesita un servicio HTTP pequeño, con carga muy concentrada (la corrida nocturna y consultas esporádicas de día) y un equipo que no quiere administrar servidores.

Opción	Ejemplo	A favor	En contra	Decisión
IaaS	Máquina virtual (Compute Engine, EC2)	Control total	Hay que parchar el sistema operativo, instalar Docker, configurar HTTPS y pagar la VM encendida 24 horas	Descartada
PaaS de contenedores	Google Cloud Run, Azure Container Apps, Render	Usa la misma imagen que ya probamos, HTTPS incluido, escala a cero, cobro por uso	Arranque en frío de unos segundos tras inactividad	Elegida
FaaS	Cloud Functions, AWS Lambda	Pago por invocación	Habría que reescribir la API en funciones y cargar el modelo en cada arranque	Descartada
SaaS de ML	Vertex AI, SageMaker endpoints	Monitoreo de modelos integrado	Costo mayor y más curva de aprendizaje para un solo modelo pequeño	Para una fase futura

Tabla 6. Comparación de opciones de despliegue.

La razón de fondo para elegir PaaS de contenedores es que el artefacto que se valida en local es exactamente el que corre en la nube. No hay una segunda configuración que pueda fallar distinto.

Modelo de implementación

Los datos que usa el servicio son ventas agregadas por día, SKU y sucursal. No incluyen nombres de pacientes, recetas ni otros datos personales, así que una nube pública es aceptable. En producción real el ERP de la cadena probablemente seguiría en su propio centro de datos, lo que da un esquema híbrido. El ERP en casa llama por HTTPS al servicio en la nube durante la noche. Si el área legal pidiera que ningún dato de ventas salga de la red privada, la misma imagen se puede correr en un clúster interno (por ejemplo, OpenShift o Kubernetes) sin cambiar el código.

Proveedor y flujo de entrega

Se eligió Google Cloud Run como opción principal porque acepta cualquier imagen de contenedor, da una URL HTTPS al desplegar, escala de cero a varias instancias según las peticiones y cobra solo mientras atiende solicitudes. Render y Azure Container Apps quedan como alternativas con el mismo enfoque.

El flujo completo queda así:

1. El desarrollador hace push a la rama main en GitHub.
2. GitHub Actions corre las 45 pruebas con pytest.
3. Si pasan, construye la imagen, la arranca, espera el estado healthy y corre la prueba de humo.
4. Si todo sale bien, publica la imagen en GHCR con las etiquetas 1.0.0 y latest.
5. Después, de forma manual (sección 7), la imagen se copia a Artifact Registry de Google Cloud y se despliega con gcloud run deploy. Cada despliegue crea una revisión nueva y se puede regresar a la anterior con un comando.

7.- Despliegue en Google Cloud Run paso a paso

Los comandos a continuación usan `PROYECTO` como identificador del proyecto de Google Cloud y la región `us-central1`. Se puede usar otra región. Conviene la más cercana que tenga Cloud Run disponible. Los comandos están en sintaxis bash. Se recomienda ejecutarlos en Google Cloud Shell, que ya incluye gcloud y Docker y no requiere gcloud auth login. Este procedimiento se ejecutó el 01/Nov/2026 en el proyecto demanda-ia.

a) Preparar el proyecto

```
gcloud auth login
gcloud config set project PROYECTO
gcloud services enable run.googleapis.com artifactregistry.googleapis.com
gcloud artifacts repositories create demanda-ia \
    --repository-format=docker --location=us-central1 \
    --description="Imagenes de DEMANDA-IA"
gcloud auth configure-docker us-central1-docker.pkg.dev
```

b) Subir la imagen

```
docker tag demanda-ia:1.0.0 us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
docker push us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
```

Alternativa sin construir la imagen: como el pipeline de CI ya publica la imagen en GitHub Container Registry, en Cloud Shell basta con descargarla y volver a etiquetarla. Este fue el camino usado en el despliegue de esta fase.

```
docker pull ghcr.io/gerardortizle/demanda-ia:1.0.0
docker tag ghcr.io/gerardortizle/demanda-ia:1.0.0 us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
docker push us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
```

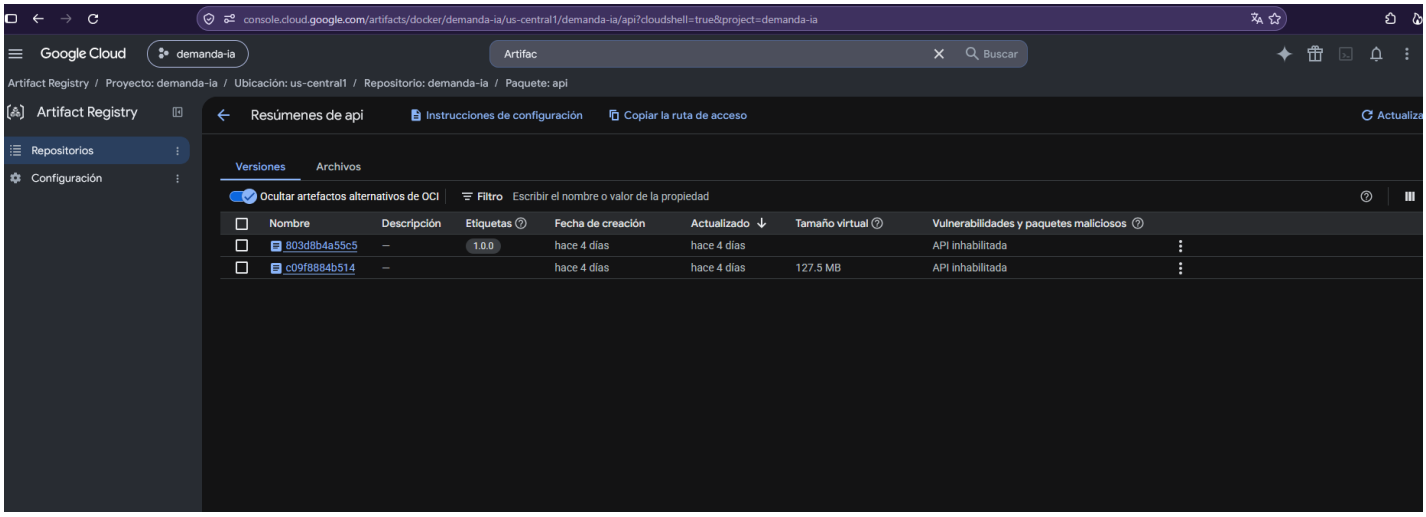


Figura 3. Imagen api:1.0.0 en el repositorio demanda-ia de Artifact Registry (us-central1)

c) Desplegar el servicio

```
gcloud run deploy demanda-ia \
    --image us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0 \
    --region us-central1 --port 8000 \
    --cpu 1 --memory 1Gi --concurrency 40 \
    --min-instances 0 --max-instances 3 \
    --set-env-vars LOG_LEVEL=INFO,MAX_LOTE=1000 \
    --allow-unauthenticated
```

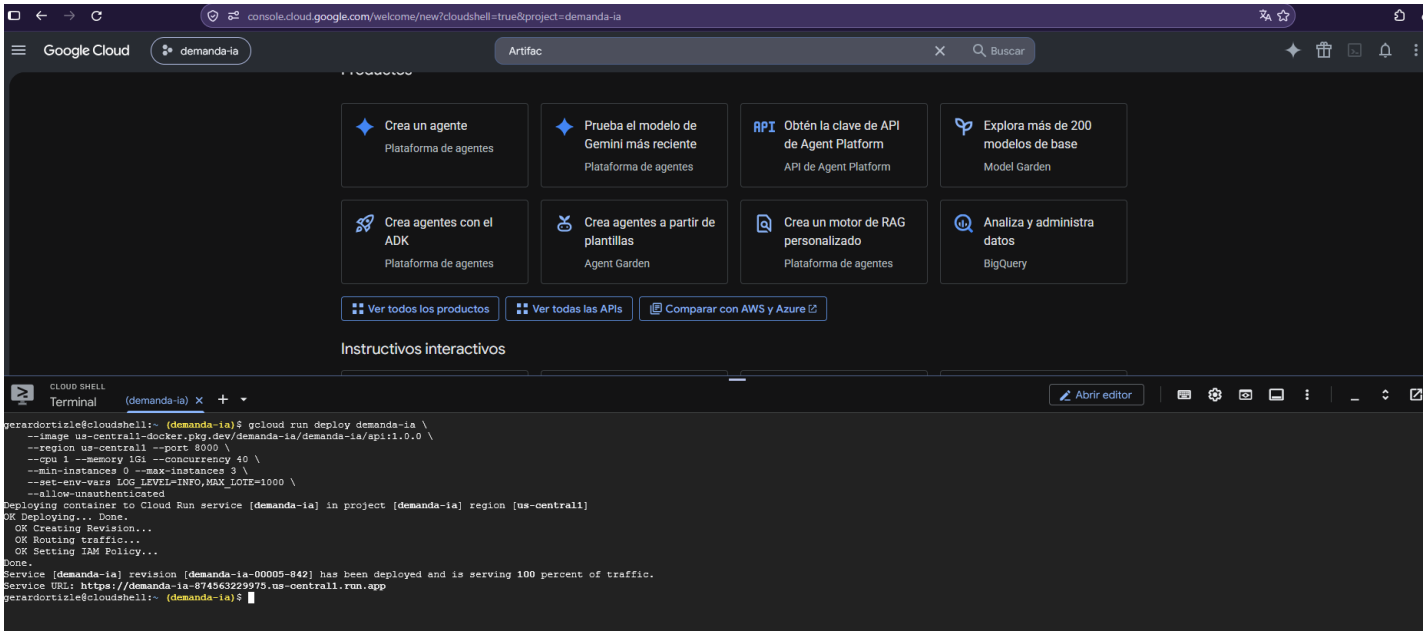


Figura 4. Salida de gcloud run deploy en Cloud Shell con la URL pública del servicio.

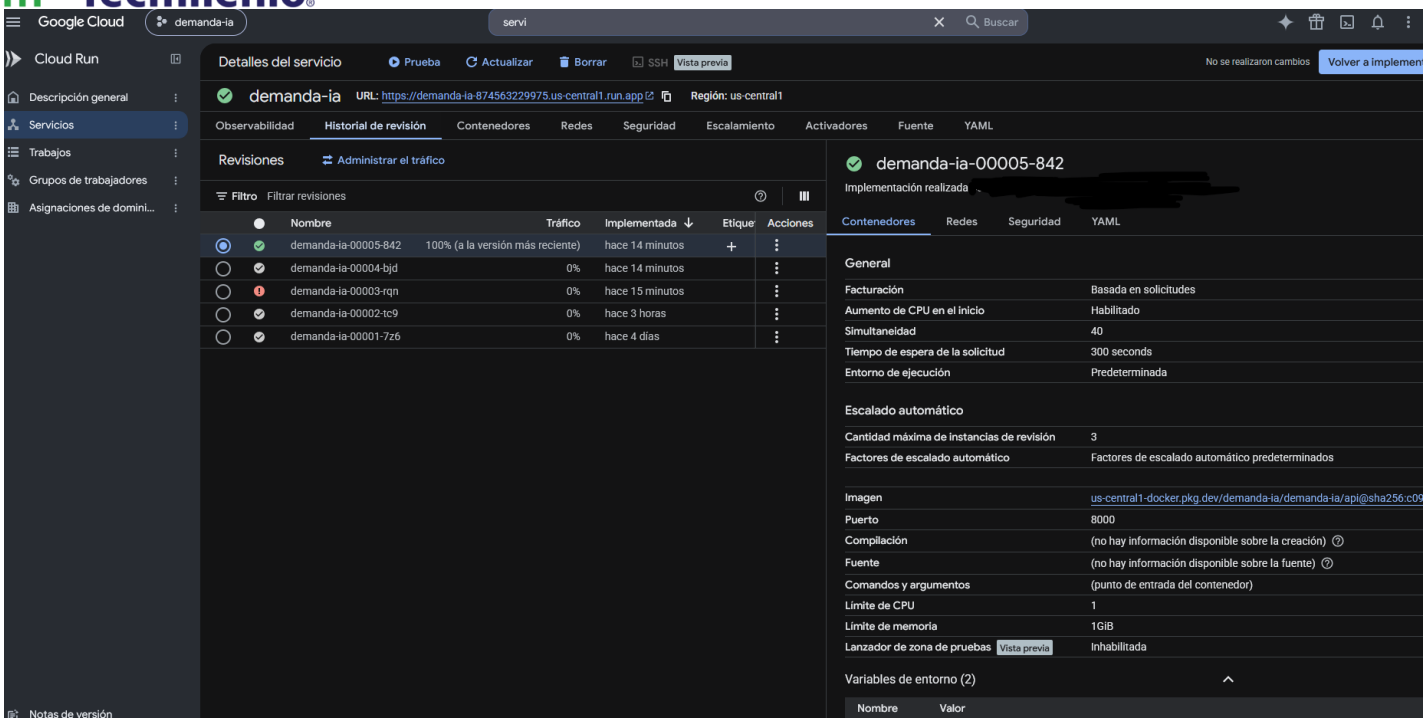


Figura 5. Servicio demanda-ia en Cloud Run (us-central1): revisión demanda-ia-00005-842 sirviendo el 100 % del tráfico, con 1 CPU, 1 GiB, simultaneidad 40 y máximo 3 instancias.

Cloud Run le pasa al contenedor el puerto indicado en `--port` mediante la variable `PORT`, y el CMD de la imagen la usa. Con `--min-instances 0` el servicio no cuesta nada mientras no recibe tráfico. A cambio, la primera petición después de un rato tarda unos segundos en lo que arranca una instancia y carga el modelo. Si eso molesta a los usuarios de día, se sube a 1.

Seguridad. `--allow-unauthenticated` deja la URL pública. No debe dejarse para el ambiente de producción y dar acceso solo a la cuenta de servicio del ERP con roles/run.invoker.

d) Verificar en la nube

```
git clone https://github.com/gerardortizle/demanda-ia.git && cd demanda-ia
URL=$(gcloud run services describe demanda-ia --region us-central1 --format='value(status.url)')
./scripts/smoke_test.sh $URL
python scripts/benchmark.py $URL 100
```

El servicio quedó disponible en https://demanda-ia-874563229975.us-central1.run.app, con la documentación interactiva en https://demanda-ia-874563229975.us-central1.run.app/docs. La primera petición puede tardar unos segundos por el arranque en frío.

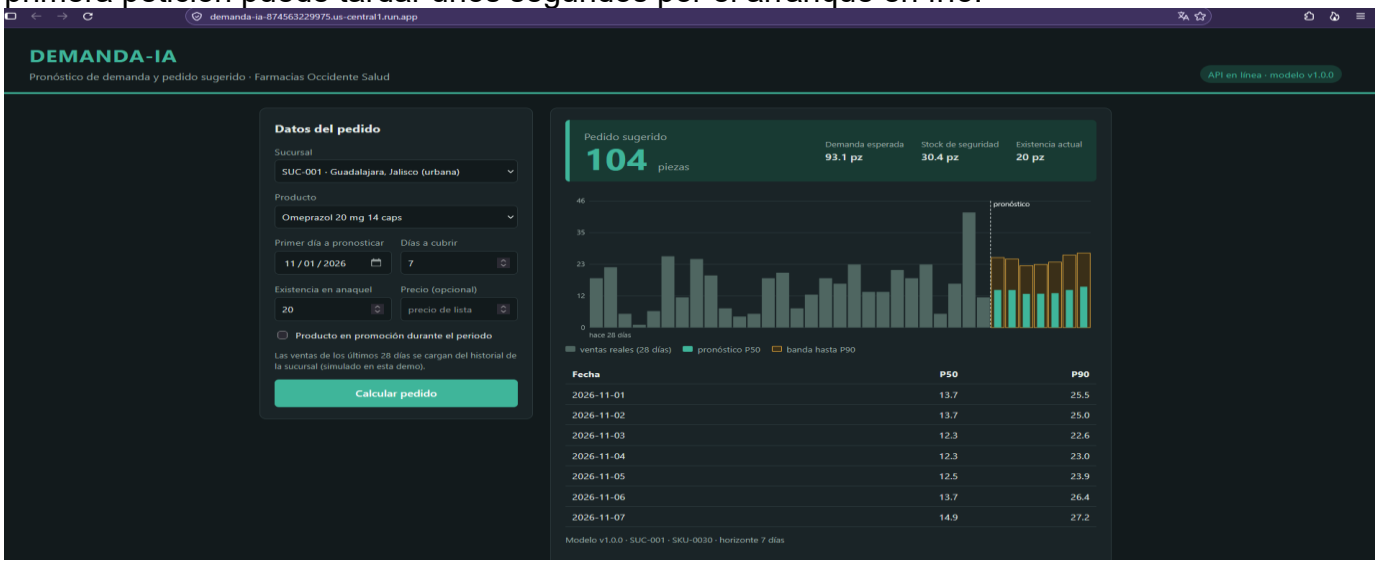


Figura 6. Interfaz web servida desde Google Cloud Run por HTTPS: pedido sugerido de 104 piezas para Omeprazol 20 mg en SUC-001.

e) Programar la corrida nocturna

En producción el ERP llamaría a `/api/v1/pronostico/lote` cada noche. Si se quiere que la nube dispare el proceso, se usa Cloud Scheduler para invocar un job (por ejemplo, un pequeño script en Cloud Run Jobs) que lea las ventas del día, arme los lotes de 1000 registros y escriba el pedido sugerido de vuelta en el ERP. Esa integración depende del ERP real y queda fuera del alcance de esta fase.

f) Actualizar y regresar versiones

```
# nueva versión del modelo o del código
docker build -t demanda-ia:1.1.0 . && docker tag ... && docker push ...
gcloud run deploy demanda-ia --image .../api:1.1.0 --region us-central1

# si algo sale mal, regresar el 100 % del tráfico a la revisión anterior
gcloud run revisions list --service demanda-ia --region us-central1
gcloud run services update-traffic demanda-ia --region us-central1 \
    --to-revisions demanda-ia-00001-abc=100
```

g) Alternativa: Render

1. En Render, crear un Web Service y elegir desplegar desde el repositorio de GitHub (Render detecta el Dockerfile) o desde la imagen `ghcr.io/gerardortizle/demanda-ia:latest`.
2. En Health Check Path escribir /api/v1/salud.
3. No hace falta configurar el puerto: Render define la variable PORT y la imagen la respeta.
4. Al terminar el despliegue, correr `./scripts/smoke_test.sh https://demanda-ia.onrender.com` con la URL que asigne Render.

En planes gratuitos el servicio se duerme tras un tiempo sin tráfico y la primera petición tarda más. Para la presentación del proyecto conviene abrir la URL unos minutos antes.

8.- Operación y mantenimiento

Variables de entorno

Variable	Valor por defecto	Descripción
PORT	8000	Puerto en el que escucha Uvicorn
WEB_CONCURRENCY	1	Número de workers. En Cloud Run conviene 1 y escalar con instancias
LOG_LEVEL	INFO	Nivel de detalle de los logs
CORS_ORIGINS	*	Dominios que pueden llamar a la API desde un navegador. En producción, la lista exacta
MAX_LOTE	1000	Máximo de registros por llamada a /pronostico/lote
MODEL_PATH	/app/models/demanda_ia_v1.joblib	Ruta del artefacto, por si se monta otro modelo

Tabla 7. Configuración del contenedor.

Monitoreo

- **Disponibilidad.** `/api/v1/salud` se puede usar como verificación de tiempo activo en Cloud Monitoring o en cualquier servicio de monitoreo externo.

- **Latencia.** Cada petición queda en el log con método, ruta, código y milisegundos, y la respuesta lleva el encabezado `X-Tiempo-Respuesta-ms`. En Cloud Run esos logs llegan solos a Cloud Logging.
- **Calidad del pronóstico.** Este es el punto que más importa y el que no se resuelve con infraestructura. Propongo comparar cada semana la venta real contra el P50 pronosticado y alertar si el WAPE semanal pasa de 30 % (la prueba dio 20 %) o si la cobertura del P90 cae debajo de 85 %.
- **Advertencias.** El campo `advertencias` señala historiales en cero, picos atípicos y precios fuera de rango. Si aparecen en muchas respuestas, algo cambió en los datos de entrada.

Reentrenamiento

Se propone reentrenar una vez al mes o cuando el monitoreo marque degradación. El procedimiento es correr `python -m demanda_ia.train`, revisar que las pruebas de `test_modelo.py` sigan pasando (comparan contra los baselines y revisan la calibración del P90), subir la versión en `config.py` y publicar una imagen con etiqueta nueva. El artefacto anterior queda en la imagen previa, así que regresar a él es cuestión de mover el tráfico.

Seguridad

- El contenedor corre sin privilegios de root y la imagen final no incluye herramientas de compilación.
- No hay secrets en la imagen ni en el repositorio. `.env` está en `.gitignore`.
- La API valida todo lo que recibe y rechaza campos extra, tipos incorrectos y valores fuera de rango con código 422. Los errores no controlados responden 500 sin mostrar detalles internos.
- Lo que queda pendiente para producción es la autenticación (IAM de Cloud Run o API gateway con llaves), CORS restringido y escaneo de vulnerabilidades de la imagen (Artifact Registry lo ofrece).

Solución de problemas frecuentes

Síntoma	Causa probable	Acción
docker build falla en pip install	Sin internet o proxy corporativo	Revisar la conexión o configurar el proxy en Docker Desktop
Error «port is already allocated»	Otro proceso usa el 8000	Usar -p 8080:8000 y abrir http://localhost:8080
docker ps muestra unhealthy	El modelo no cargó	Ver docker logs demanda; revisar que exista models/demanda_ia_v1.joblib
Advertencia de versión de scikit-learn al cargar	Se cambió requirements.txt sin reentrenar	Regresar a la versión fija o reentrenar
El frontend dice «Sin conexión con la API»	Frontend y API en dominios distintos	Definir window.DEMANDA_API y agregar el dominio a CORS_ORIGINS
En la nube la primera petición tarda	Arranque en frío con min-instances 0	Subir --min-instances a 1
Respuesta 422	Entrada inválida	Leer el campo detail: indica el campo y el motivo
ALREADY_EXISTS al crear el repositorio de Artifact Registry	El repositorio se creó en un intento anterior	Ignorar el error y continuar; se reutiliza el repositorio existente
gcloud run deploy falla con «Image … not found»	Se dejó el texto PROYECTO literal en la ruta de la imagen	Reemplazar PROYECTO por el ID real del proyecto y volver a desplegar

Tabla 8. Problemas frecuentes.

9.- Alineación con ISO/IEC 23053

La norma ISO/IEC 23053:2022 describe el ciclo de vida de un sistema de IA basado en aprendizaje automático. No obliga a usar un algoritmo o arquitectura en particular, pero pide que cada etapa tenga evidencia. Esta tabla relaciona cada etapa con lo que existe en el repositorio:

Etapa	Evidencia en DEMANDA-IA
Propósito y requisitos	Caso de negocio de la Fase I; sección 1 de este manual
Adquisición y preparación de datos	Simulador data.py con patrones documentados; en producción, extracción del ERP
Ingeniería de variables	features.py, compartido entre entrenamiento e inferencia, con prueba que verifica que coinciden
Entrenamiento	train.py, separado del servicio, con semilla fija y metadatos guardados junto al modelo
Verificación y validación	45 pruebas automáticas y el Documento de validación y pruebas
Despliegue	Dockerfile, CI/CD y este manual
Operación y monitoreo	Endpoint de salud, logs con latencia, advertencias y propuesta de umbrales (sección 8 Monitoreo)
Retiro	Versionado de imágenes y revisiones de Cloud Run; para retirar el modelo se elimina el servicio y se conserva la imagen etiquetada como registro

Tabla 9. Etapas del ciclo de vida y evidencias.

10.- Uso de herramientas de IA generativa en la documentación

Como se propone en el Tema 16, de usar IA generativa para acelerar la documentación técnica, en este proyecto así se usó, y está documentado por escrito ya que creo que es parte de documentar con honestidad:

- El código, las pruebas y el primer borrador de este manual y del documento de validación se elaboraron con apoyo de Claude (Anthropic). Las cifras de las tablas salen de la ejecución real de los scripts del repositorio, no de la herramienta.

- Para revisar la estructura contra la norma usé Microsoft Copilot con el siguiente prompt, en la línea del ejemplo del curso:

```
Actúa como especialista en documentación de sistemas de IA. Revisa el manual de
despliegue adjunto del proyecto DEMANDA-IA (pronóstico de demanda por SKU y sucursal
para una cadena de farmacias, API FastAPI en Docker desplegada en Google Cloud Run).
Compáralo con las etapas del ciclo de vida de ISO/IEC 23053:2022 e indica qué
secciones faltan o están incompletas en: propósito, datos, verificación y validación,
despliegue, monitoreo y retiro. Entrega una tabla con hallazgo, sección y recomendación.
```

Bibliografía

Chen, M. (2025). *¿Qué es una API (interfaz de programación de aplicaciones)? Oracle.* Obtenido de https://www.oracle.com/latam/cloud/cloud-native/api-management/what-is-api/

China, C. y Goodwin, M. (s.f.). *IaaS, PaaS, SaaS: ¿cuál es la diferencia? IBM.* Obtenido de https://www.ibm.com/mx-es/think/topics/iaas-paas-saas

Docker Inc. (s.f.). *Dockerfile reference.* Obtenido de https://docs.docker.com/reference/dockerfile/

FastAPI. (s.f.). *FastAPI in Containers - Docker*. Obtenido de https://fastapi.tiangolo.com/deployment/docker/

Google Cloud. (s.f.). *Cloud Run documentation.* Obtenido de https://cloud.google.com/run/docs

ISO/IEC. (2022). *ISO/IEC 23053:2022 Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). International Organization for Standardization.*

Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*

Universidad Tecmilenio. (2026). *Tema 15. Contenerizar el modelo con Docker [material de curso]*. Obtenido de https://cursos.tecmilenio.mx/courses/252869

Universidad Tecmilenio. (2026). *Tema 16. Despliegue en la nube [material de curso].* Obtenido de https://cursos.tecmilenio.mx/courses/252869